

MapReduce

Dongchul Park

Chung-Ang University

Large scale computing for big data

- **Challenges:**

- How to distribute computation?
- Distributed/parallel programming is hard

- **MapReduce** addresses all of the above

- Google's computational/data manipulation model
- Elegant way to work with big data

MapReduce

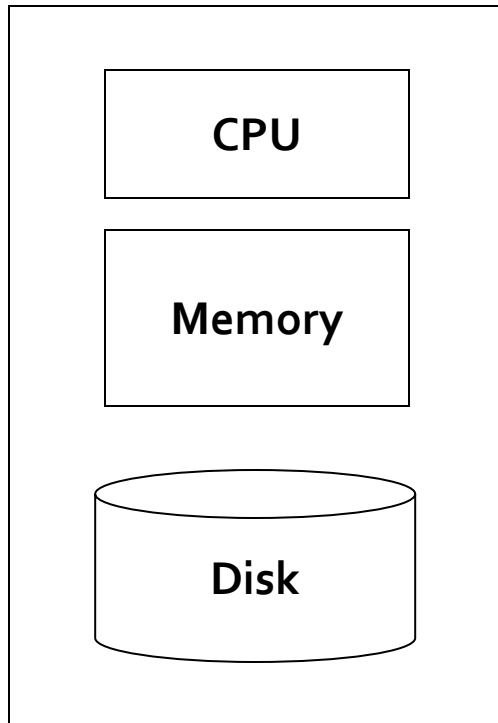
- “A simple and powerful interface that enables automatic parallelization and distribution of large-scale computations, combined with an implementation of this interface that achieves high performance on large clusters of commodity PCs.”

Dean and Ghemawat, “MapReduce: Simplified Data Processing on Large Clusters”, Google Inc.

MapReduce

- More simply, MapReduce is
 - A parallel programming model and associated implementation.
 - Process data using special *map()* and *reduce()* functions.

Single Node Architecture

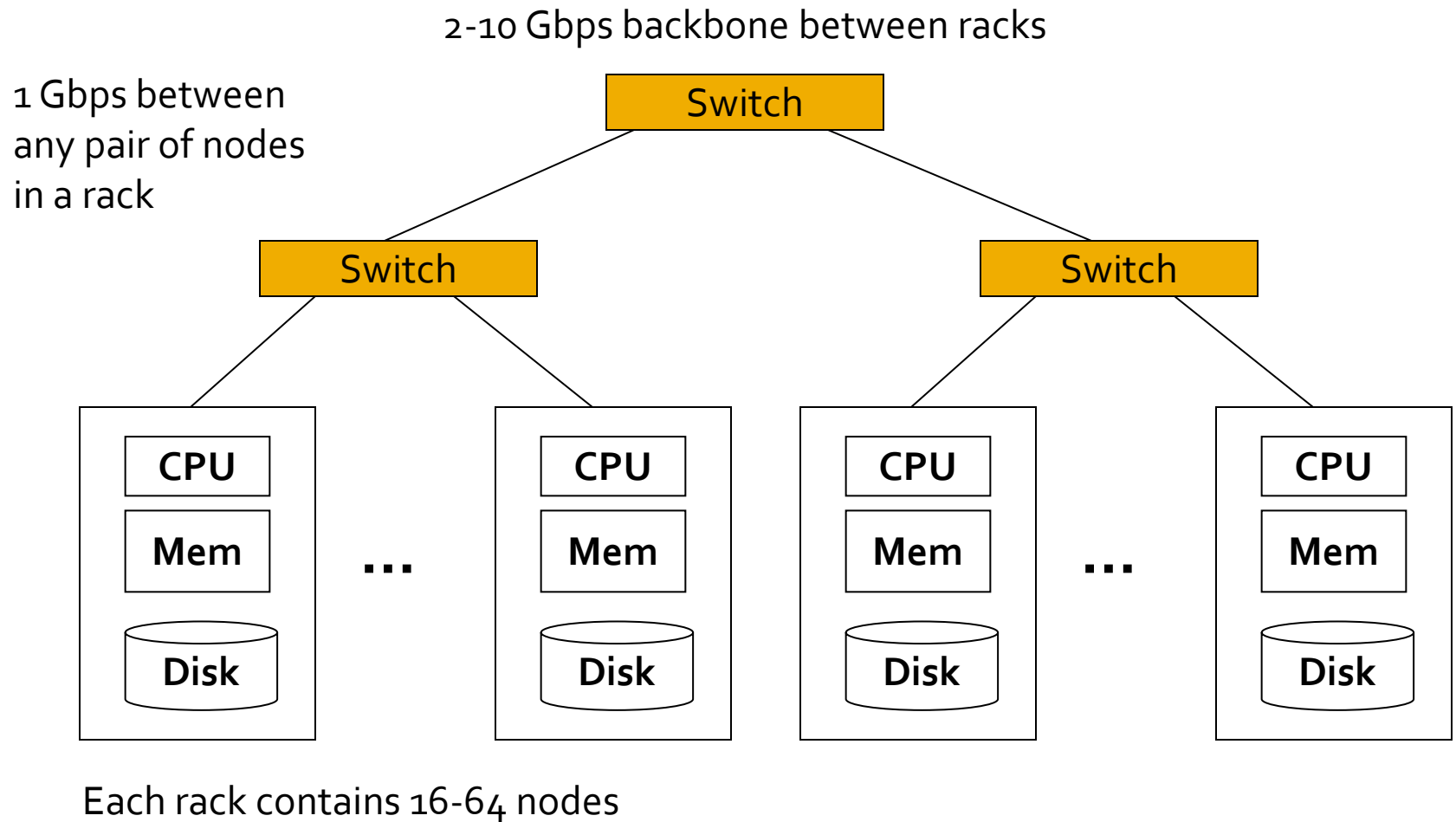


**“Classical” Data Processing
Architecture**

Motivation: Google Example

- 20+ billion web pages x 20KB = 400+ TB
- 1 computer reads 30-35 MB/sec from disk
 - ~4 months to read the web
- ~1,000 HDDs to store the web
- Takes even more to **do** something useful with the data!
- Today, a standard architecture for such problems is emerging:
 - Cluster of commodity Linux nodes
 - Commodity network (Ethernet) to connect them

Cluster Architecture





Large-scale Computing

- **Large-scale computing on commodity hardware**
- **Challenges:**
 - **How do you distribute computation?**
 - **How can we make it easy to write distributed programs?**
 - **How to deal with machine failure?**
 - One server may stay up 3 years (1,000 days)
 - If you have 1,000 servers, expect to loose 1/day
 - People estimated Google had ~1M machines in 2011
 - 1,000 machines fail every day!

Idea and Solution

- **Issue:** Copying data over a network takes long time
- **Key Ideas:**
 - Bring computation close to the data
 - Store files multiple times for reliability (replication)
- **MapReduce** addresses these problems
 - **(1) Storage Infrastructure – File system**
 - Google GFS, Hadoop HDFS
 - **(2) Programming model**
 - MapReduce

(1) Storage Infrastructure

■ Problem:

- If nodes fail, how to store data persistently?

■ Answer:

■ Distributed File System (DFS):

- Provides global file namespace
- E.g.,) Hadoop HDFS;

■ Typical usage pattern

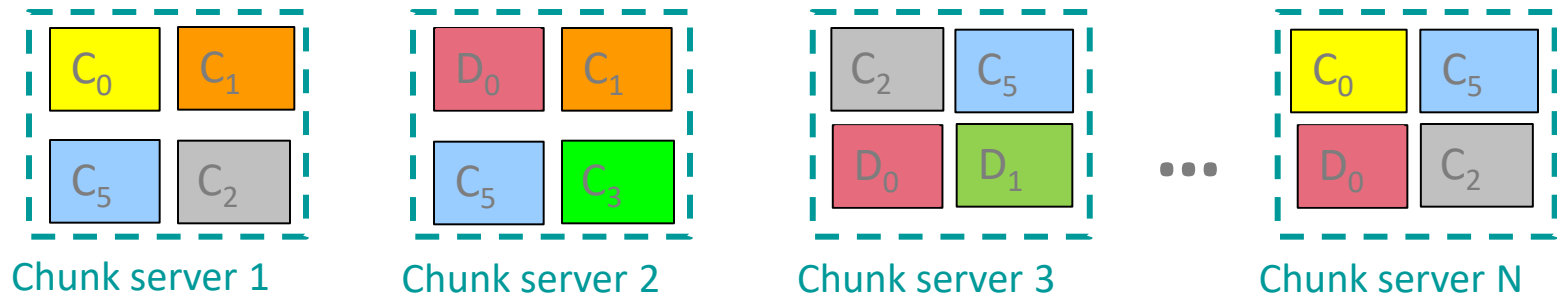
- Huge files (100s of GB to TB)
- WORM (Write Once Read Many)
 - Data is rarely updated in place
 - Reads and appends are common

Distributed File System (DFS)

- **Data nodes (or, chunk servers)**
 - File is split into contiguous chunks
 - Typically each chunk is 16-64MB
 - Each chunk replicated (usually 2x or 3x: replication factor)
 - Try to keep replicas in different racks for reliability
- **Master node**
 - Stores metadata about where files are stored
 - E.g., Name Node in Hadoop's HDFS
 - Might be replicated
- **Client library for file access**
 - Talks to master to find chunk servers
 - Connects directly to chunk servers to access data

Distributed File System (DFS)

- **Reliable distributed file system**
- Data kept in “chunks” spread across machines
- Each chunk **replicated** on different machines
 - Seamless recovery from disk or machine failure



Bring computation directly to the data!

Chunk servers also serve as compute servers

(2) Programming Model: MapReduce

Warm-up task:

- We have a huge text document
- Count the number of times each distinct word appears in the file
- **Sample application:**
 - Analyze web server logs to find popular URLs

Task: Word Count

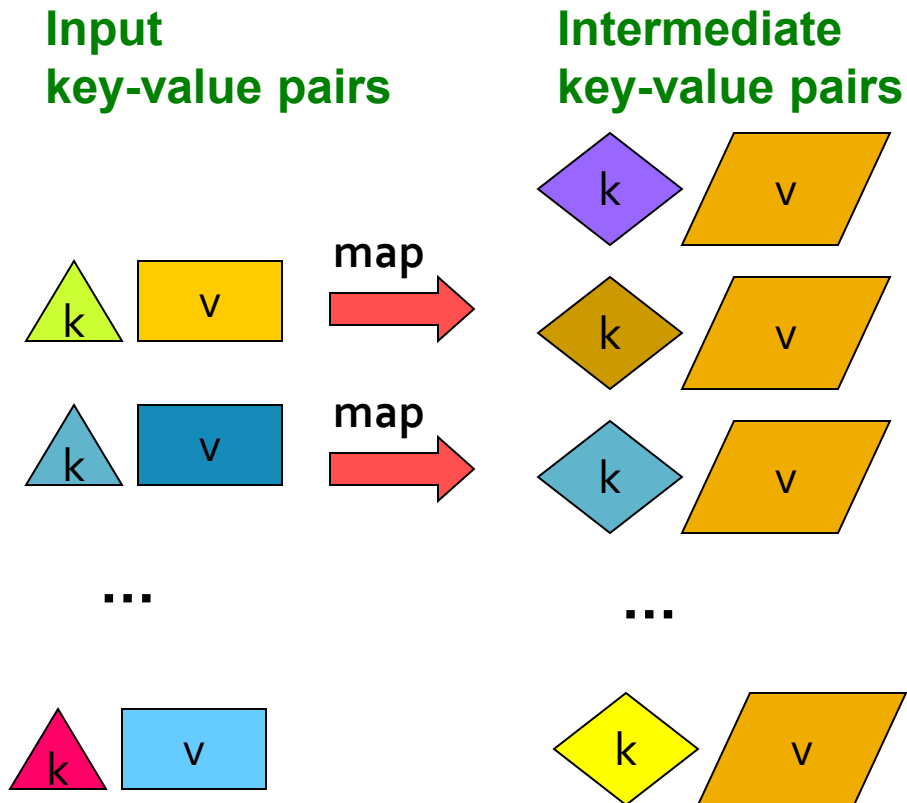
- Count occurrences of words:
 - `words (doc.txt) | sort | uniq -c`
 - where **words** takes a file and outputs the words in it, one per a line
- This captures the essence of **MapReduce**
 - Great thing is that it is naturally parallelizable

MapReduce: overall procedures

- 1. Sequentially read a lot of data
- 2. **Map:**
 - Extract something you care about
- 3. **Group by key:** Sort and Shuffle
- 4. **Reduce:**
 - Aggregate, summarize, filter or transform
- 5. Write the result

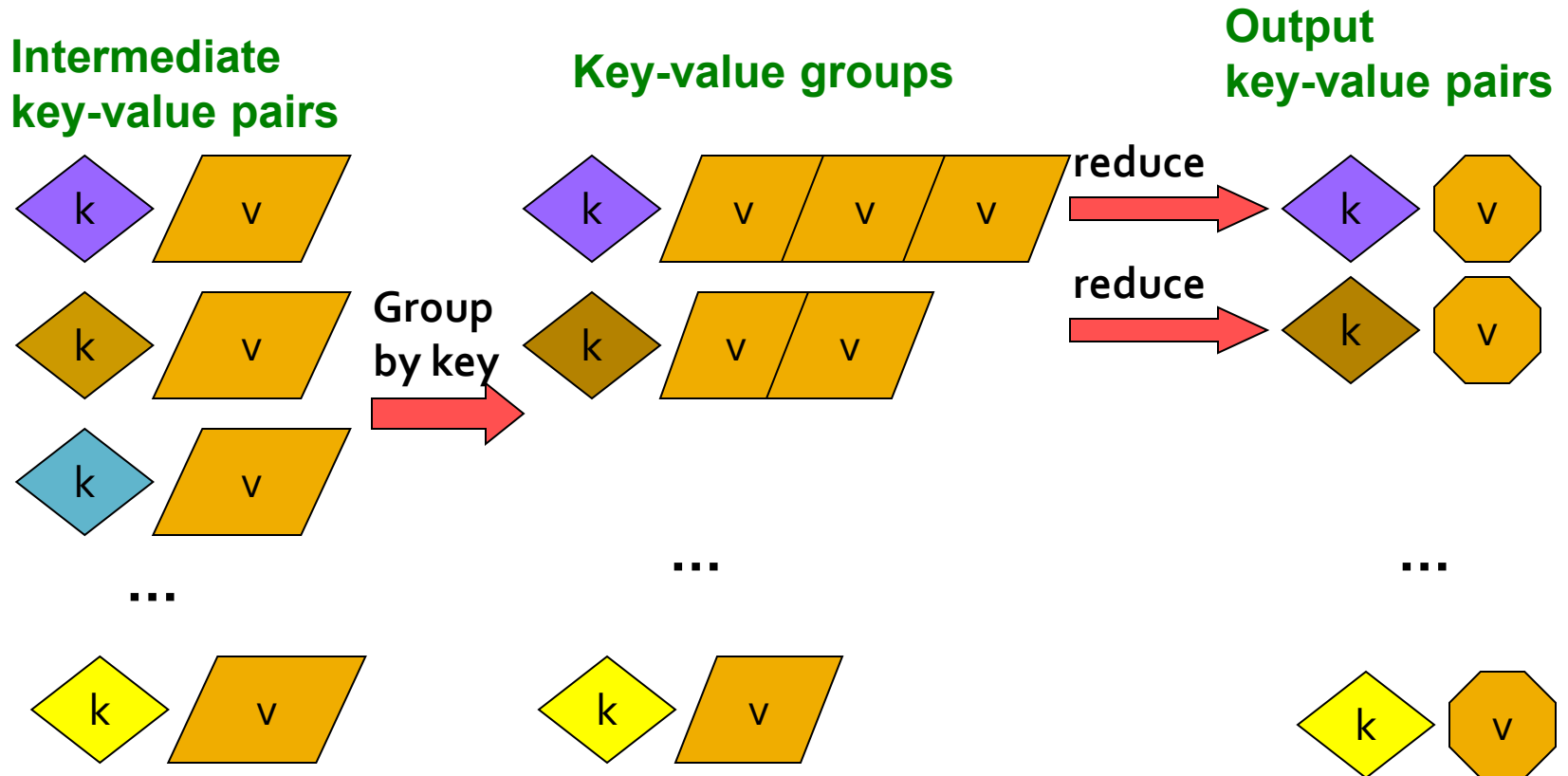
MapReduce: logical view

Map step

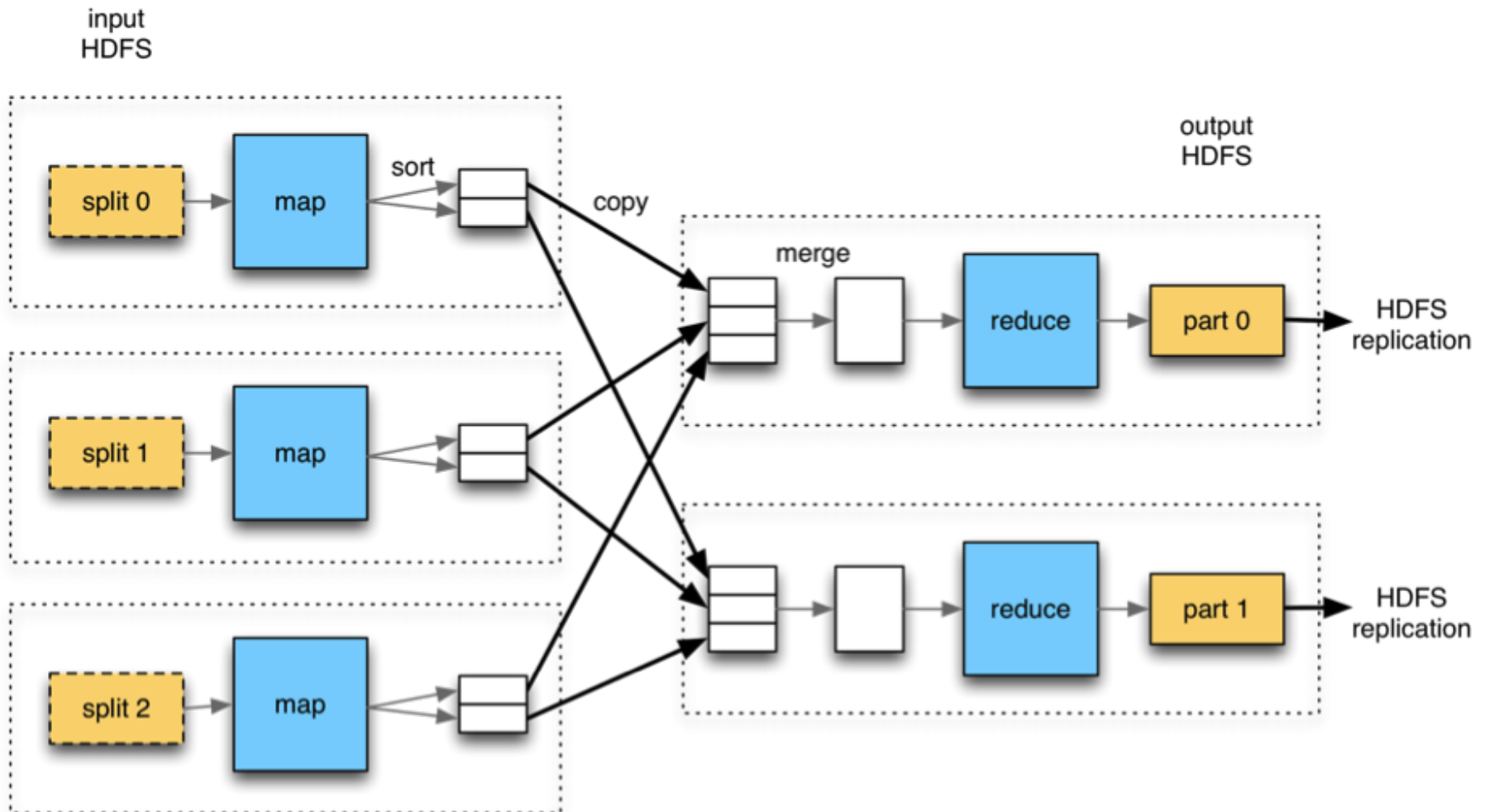


MapReduce: logical view

Reduce step



MapReduce: physical view

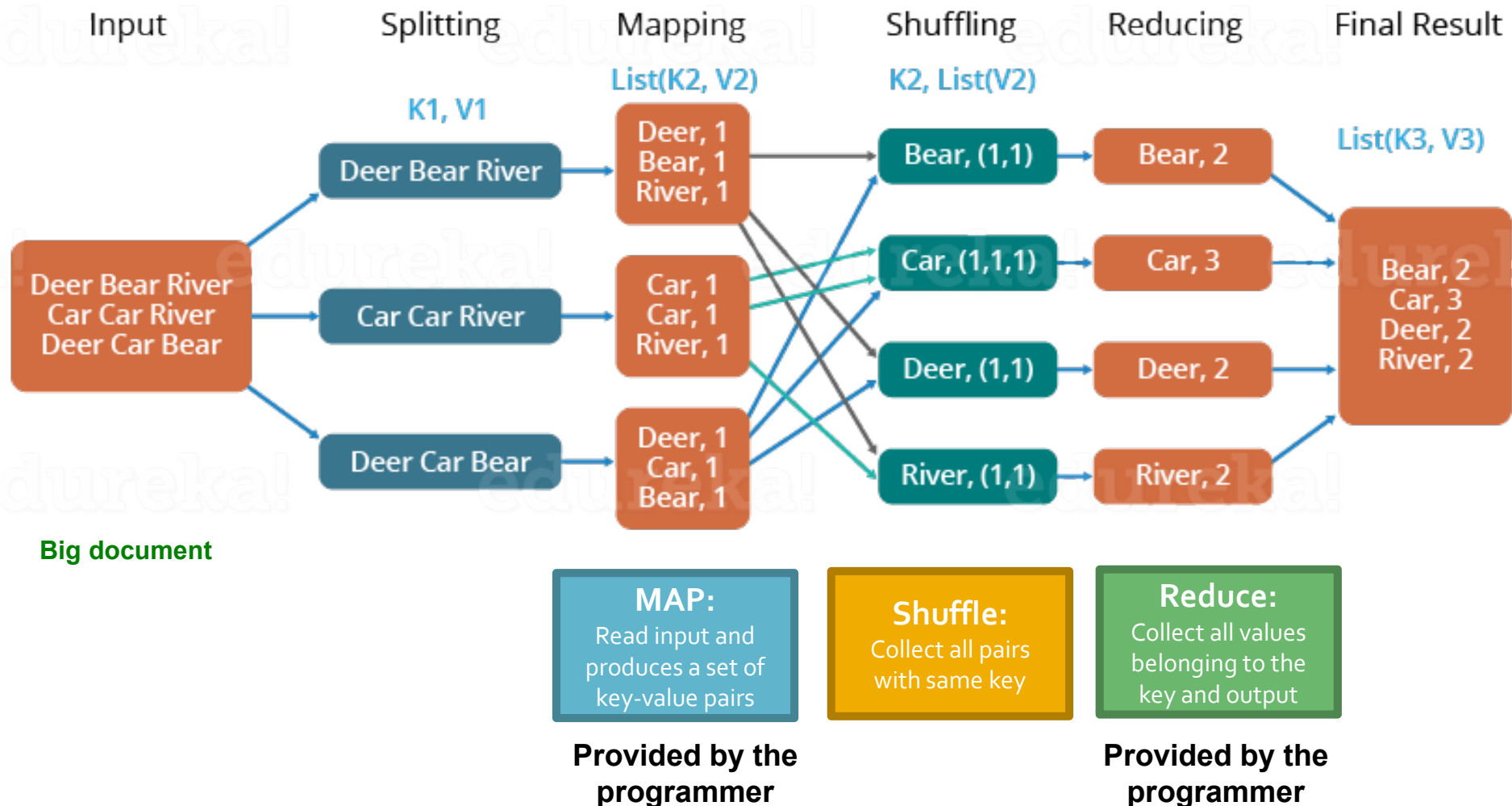


More Specifically

- **Input:** a set of key-value pairs
- Programmer specifies two methods:
 - **Map(k, v)** $\rightarrow \langle k', v' \rangle^*$
 - Takes a key-value pair and outputs a set of key-value pairs
 - E.g., key is the filename, value is a single line in the file
 - **Reduce($k', \langle v' \rangle^*$)** $\rightarrow \langle k', v'' \rangle^*$
 - All values v' with same key k' are reduced together and processed in k' order

MapReduce: Word Count

The Overall MapReduce Word Count Process



Word Count Using MapReduce

map(key, value) :

/ key: document name; value: text of the document*/*

for each word w in value:
 emit(w, 1)

reduce(key, values) :

/ key: a word; value: an iterator over counts */*

result = 0
for each count v in values:
 result += v
emit(key, result)

MapReduce: Environment

MapReduce environment takes care of:

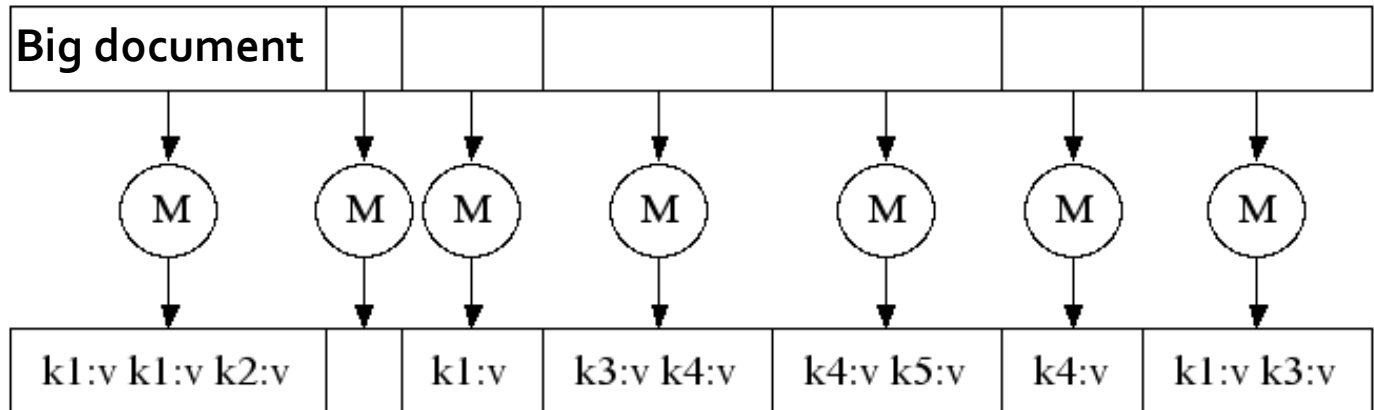
- Partitioning the input data
- Scheduling the program's execution across a set of machines
- Performing the group by key step
- Handling machine failures
- Managing required inter-machine communication

MapReduce: A diagram

MAP:

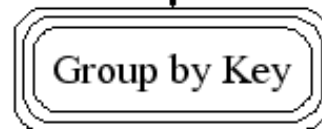
Read input and produces a set of key-value pairs

Input

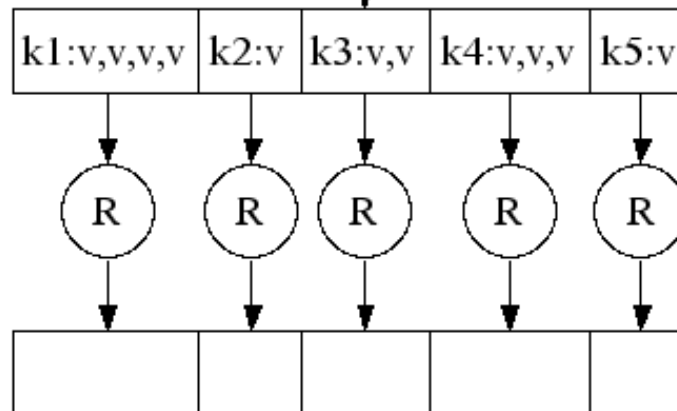


Group by key:

Collect all pairs with same key
(Hash merge, Shuffle, Sort, Partition)



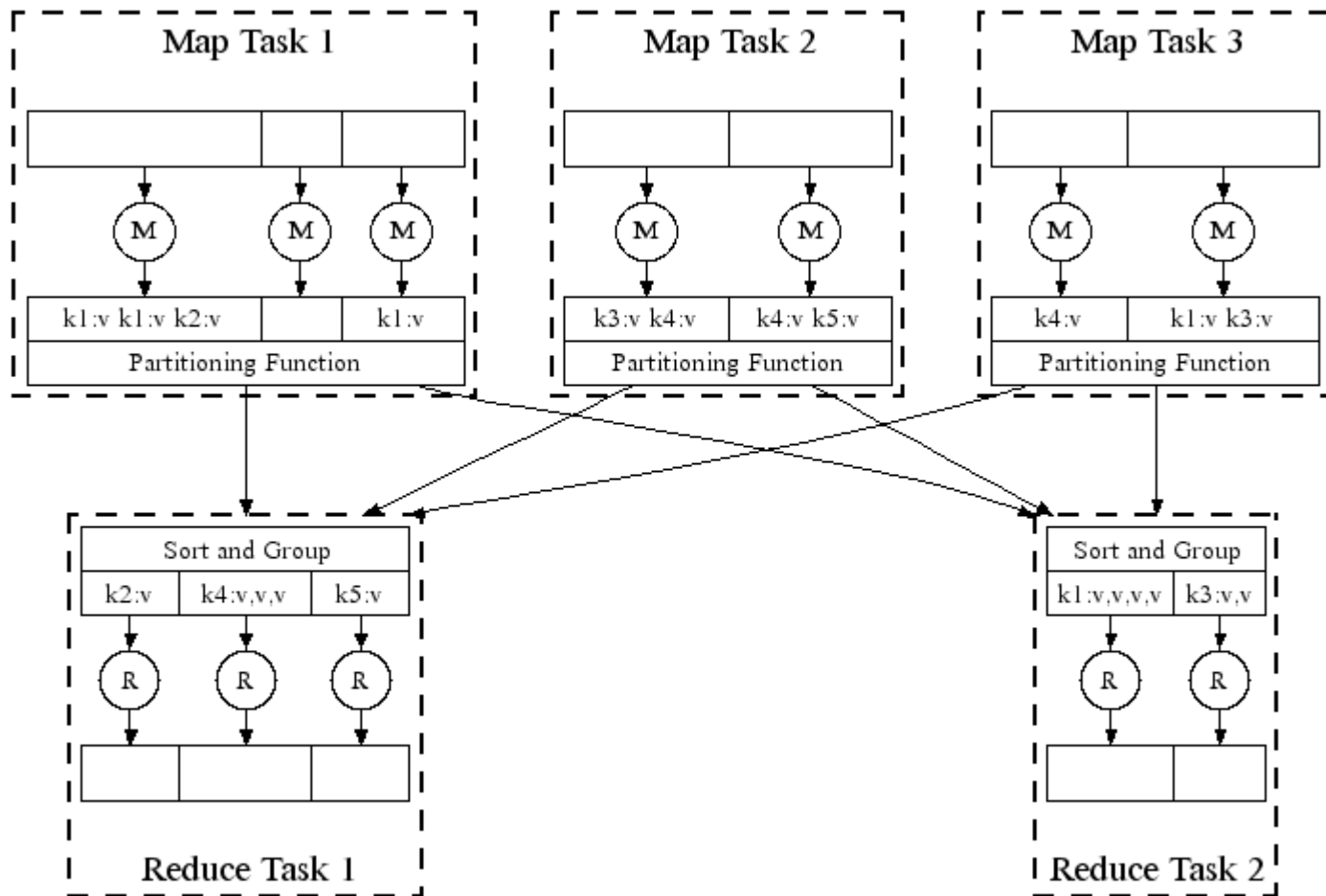
Grouped



Reduce:

Collect all values belonging to the key and output

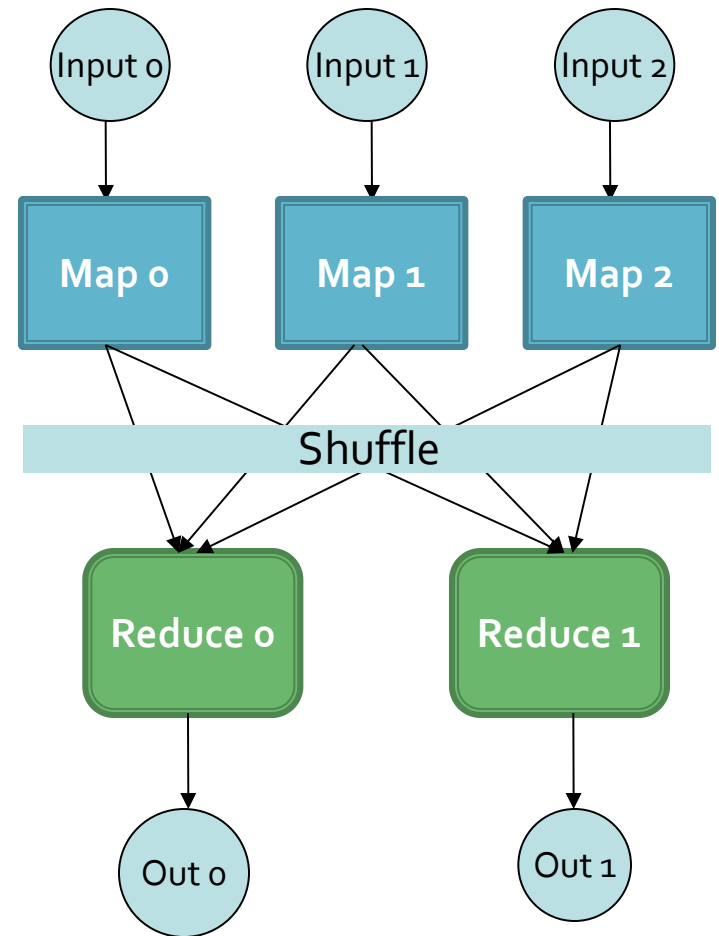
MapReduce: Parallelism



All phases are distributed with many tasks doing the work

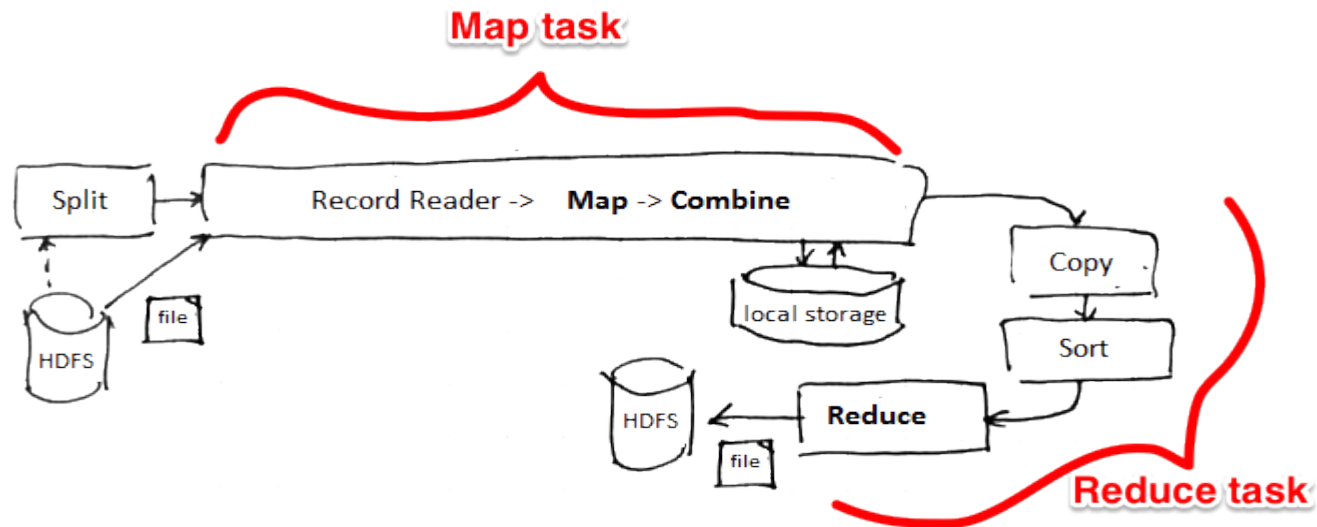
MapReduce: Workflow

- **Programmer specifies:**
 - Map and Reduce
 - Input and output format
- **Workflow:**
 - Read inputs as a set of key-value pairs
 - **Map** transforms input kv-pairs into a new set of k'v'-pairs
 - Sorts & Shuffles the k'v'-pairs to output nodes
 - All k'v'-pairs with a given k' are sent to the same **reduce**
 - **Reduce** processes all k'v'-pairs grouped by key into new k'v''-pairs
 - Write the resulting pairs to files



Data Flow

- **Input and final output are stored on a distributed file system (FS):**
 - Scheduler tries to schedule map tasks “close” to physical storage location of input data
- **Intermediate results are stored on local FS of Map and Reduce workers**



Coordination: Master

- **Master node takes care of coordination:**
 - **Task status:** (idle, in-progress, completed)
 - **Idle tasks** get scheduled as workers become available
 - When a map task completes, it sends the master the location and sizes of its R intermediate files, one for each reducer
 - Master pushes this info to reducers
- Master pings workers periodically to detect failures (i.e., heartbeat)

Dealing with Failures

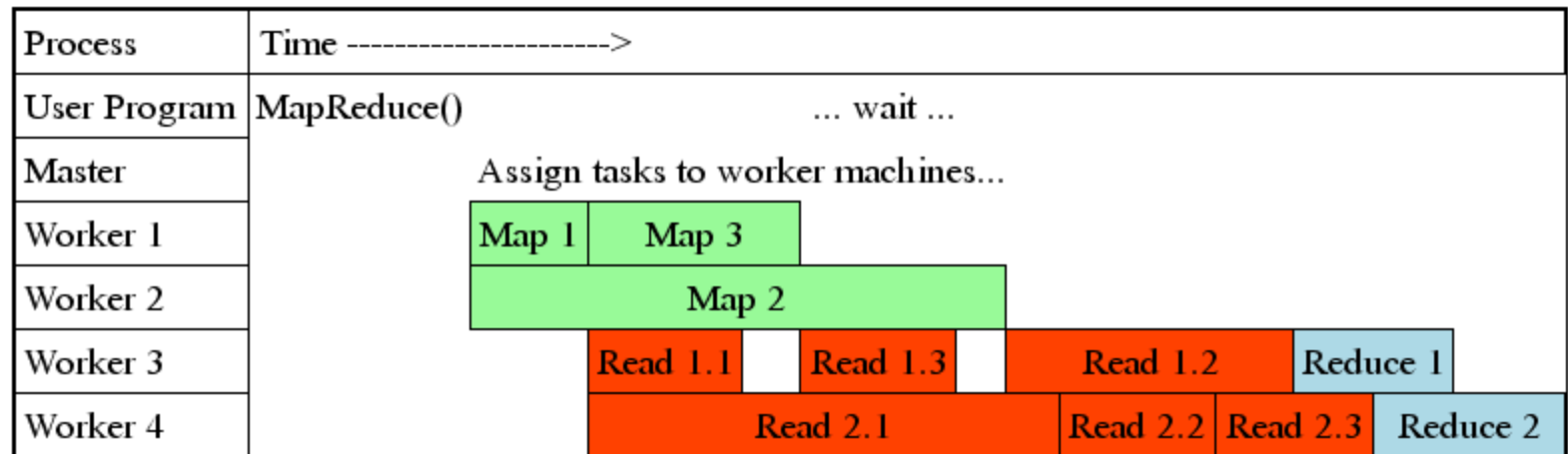
- *Re-execution is the main mechanism for fault-tolerance*
- **Worker failure**
 - Master detects worker failures via periodic heartbeats
 - The master drives the re-execution of tasks
 - Both completed and in-progress map tasks are re-executed.
 - Even the completed map tasks also need to be re-executed? Why?
 - Only in-progress reduce tasks are re-executed.
 - Completed reduces tasks do not need to be re-executed
- **Master failure**
 - The initial implementation did not support master failure
 - MapReduce task is aborted and client is notified
 - Solutions
 - Checkpoint the state of internal structures in the DFS
 - Use replication techniques
- Robust: lost 1600 of 1800 machines once, but finished fine.

How many Map and Reduce tasks?

- M map tasks, R reduce tasks
- **Rule of a thumb:**
 - Make M much larger than the number of nodes in the cluster
 - Improves dynamic load balancing and speeds up recovery from worker failures
 - One DFS chunk per map is common
- **Usually R is smaller than M**
 - Because output is spread across R files

Task Granularity & Pipelining

- **Fine granularity tasks:** map tasks $\gg$ machines
 - Minimizes time for fault recovery
 - Can do pipeline shuffling with map execution
 - Better dynamic load balancing



Note: Read $a.b$: Reducer a reads Map b output data

Refinements: Locality

- Most input data is read locally.
- Why?
 - To reduce network bandwidth consumption
- How does MapReduce achieve locality?
 - The master attempts to schedule a map task on a machine that contains a replica of the corresponding input data
 - If it fails, attempts to schedule near replica (e.g., on the same network switch)

Refinements: Backup Tasks

■ Problem

- Slow workers significantly lengthen the job completion time:
 - Other jobs on the machine
 - Bad disks, weird things

■ Solution

- Near end of phase, spawn backup copies of tasks
 - ➔ Speculative execution in Hadoop
 - Whichever one finishes first “wins”

■ Effect

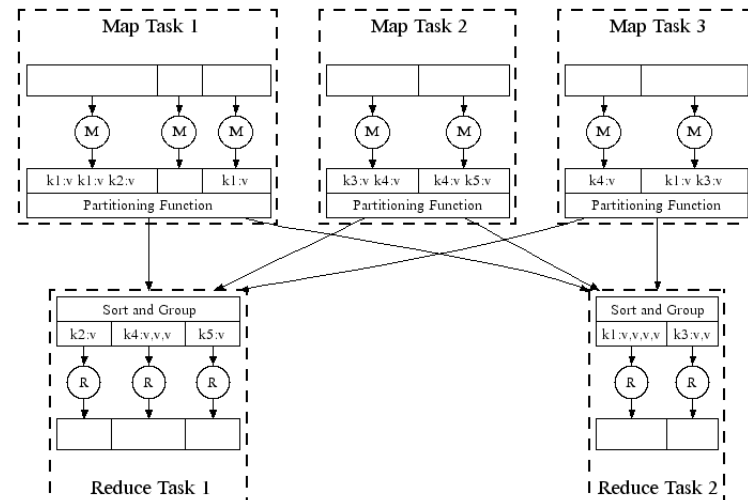
- Dramatically shortens job completion time

Refinement: Combiners

- **Issue:** Often a Map task will produce many pairs of the form $(k, v_1), (k, v_2), \dots$ for the same key k
 - E.g., popular words in the word count example

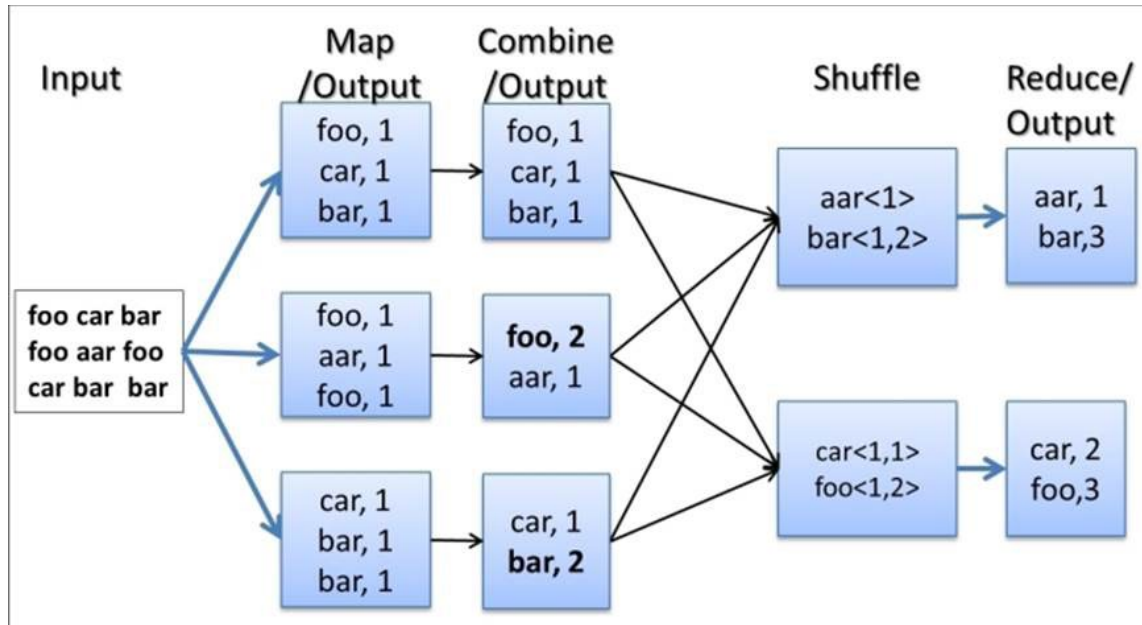
- **Can save network time by pre-aggregating values in the mapper:**

- $\text{combine}(k, \text{list}(v_1)) \rightarrow v_2$
- Combiner is usually the same as the reduce function
 - a.k.a. semi-reducer



Refinement: Combiners

- **Back to our wordcount example:**
 - Combiner combines the values of all keys of a single mapper (single machine):



- Much less data needs to be copied and shuffled over the network!

Refinement: Partitioner

- **Want to control how keys get partitioned**
 - Inputs to map tasks are created by contiguous splits of input file
 - Reduce needs to ensure that records with the same intermediate key end up at the same worker
- **System uses a default partition function:**
 - $\text{hash}(\text{key}) \bmod R$
- **Sometimes useful to override the hash function:**
 - E.g., $\text{hash}(\text{hostname}(\text{URL})) \bmod R$ ensures URLs from a host end up in the same output file

Problems Suited for MapReduce

Basic MapReduce patterns

- Counting and summing
 - Applications: log analysis, data querying
- Collating
 - Applications: inverted index, ETL (Extract, Transform, Load)
- Filtering (a.k.a. Grepping), Parsing, Validation
 - Applications: log analysis, data querying, ETL, data validation
- Distributed task execution
 - Applications: physical and engineering simulation, numerical analysis, performance testing
- Sorting
 - Applications: ETL, data analysis

Not-so-basic MR patterns

- Iterative messaging passing (graph processing)
 - E.g., machine learning, breadth-first-search, page rank
- Cross-correlation
 - E.g., market analysis, text analysis

Reading

- Jeffrey Dean and Sanjay Ghemawat, “MapReduce: Simplified Data Processing on Large Clusters,” in *OSDI'04*, San Francisco, CA, December 2004.
 - <https://static.googleusercontent.com/media/research.google.com/en//archive/mapreduce-osdi04.pdf>